Guided Soft Actor–Critic

Blind Go2-W Locomotion — Theory and Implementation

Tamer Bora İkizoğlu

Istanbul Technical University

Outline

The problem

Blind locomotion is a POMDP

In simulation we can read anything: body linear velocity, a height scan of the terrain, contact forces, friction, every randomized dynamics parameter.

On the real robot almost none of that exists. The deployed policy sees only proprioception: joint encoders, IMU, its own previous action.

The asymmetry

Training-time information $\gg$ deployment-time information. A policy trained on what it *cannot* have at deployment is useless; a policy that ignores what the simulator knows is wasteful.

Guided SAC is one answer: let a *privileged* network help train a *blind* network that is the only thing shipped.

Two observation channels

Proprioceptive, noisy, 5-frame history
5×57: base ang. vel. (3), projected gravity (3), velocity command (3), joint pos (16), joint vel (16), previous action (16)

Privileged, clean, single frame
$57 + 3 + 187$: same frame without noise, base *linear* velocity (3), height scan (187)

Action space: 16 dims = 12 leg position targets + 4 wheel velocity targets, tanh-squashed and scaled by per-dimension bounds (legs 5.0, wheels 10.0).

Guided SAC: the idea

Origin, and what we changed

Haklıdır & Temeltaş (2021) and Çerçi & Temeltaş (2024) train **two actors**: a *guiding actor* that sees clean state, and a *control actor* that sees degraded state and is pulled toward the guide's actions.

	Çerçi (2024)	**This project**
Why the control actor is handicapped	adversarial perturbation	*genuine* partial observability
Guiding actor sees	clean state s_t	privileged $\tilde{s}_t$ (height scan, lin. vel.)
Control actor sees	perturbed s_t^p	proprioception o_t + history
Shapes	identical	**different** (285 vs 532)
Agents	$k = 3, 5, 7$	$k = 1$ (single agent)

Two consequences: the paper's $s_t^p \to o_t$ mapping is *not* cosmetic (different dimensionality), and every multi-agent construction collapses at $k = 1$ — it must be re-derived, not copied.

Guided SAC is three mechanisms, not one

M1 — Asymmetric critic

$Q(\tilde{s}_t, a)$ sees everything, $\pi_f(a|o_t)$ does not

cost: *zero* extra networks
fixes credit assignment

M2 — Guide as behavior policy

the teacher also *acts* in the environment

cost: one extra actor
widens state visitation

M3 — Action-space imitation

pull π_f toward π_g with weight λ

cost: one norm term
dense low-variance target

The literature ships these **bundled**. Separating them matters: M1 is free and not specific to Guided SAC, while M2 is the expensive one. Our design keeps M1, keeps M3 in softened form, and uses only a *small* amount of M2 — for a different reason than the original (next slides).

Architecture

Vocabulary, before the diagrams

Critic (Q). A *prediction*: "if I take action a in state s, how much total reward follows?" Learned from recorded experience — it is not given.

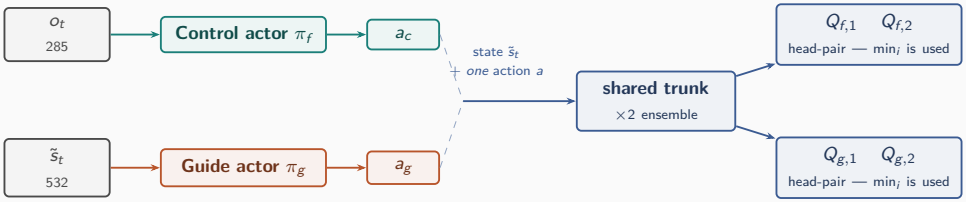
Replay buffer, and "stored a_t". Every step the robot takes is written down. *Stored* a_t means **the action that was really executed** at that moment — as opposed to a hypothetical action we ask the critic about afterwards.

Trunk and head. The **trunk** is the bulk of the critic network: it turns (s, a) into features. A **head** is a small output layer turning those features into a number. Two heads on one trunk = two different answers from one shared body.

Target network. A slowly-updated copy of the critic, used only to build the training signal — so the critic is not chasing its own constantly-moving output.

Bootstrap. The critic is trained toward *reward now* + *its own estimate of the value of the next state*. That second half is the bootstrap — and computing it forces a choice: **whose policy acts next?** That single choice is what will separate Q_f from Q_g.

Three networks, and two value functions



What the two heads mean

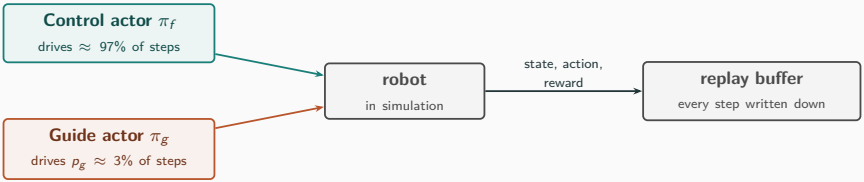
$Q_f(\tilde{s}_t, a)$ — how good is taking action a now, **if the blind student acts from then on**

$Q_g(\tilde{s}_t, a)$ — how good is taking action a now, **if the privileged teacher acts from then on**

Two trunks, and **two heads on each** — four in total. Within a pair the **smaller** value is always taken: that pessimism is the guide's only protection against its own value estimate running away.

Where the data comes from

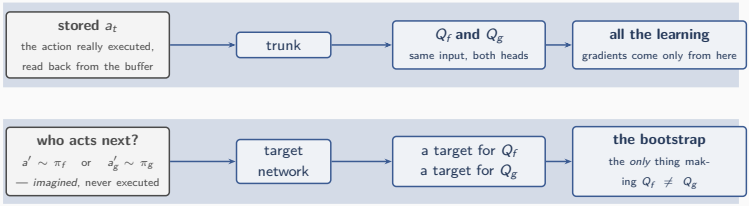
Before anything is learned, the robot must act and the experience must be recorded. **Both actors can drive the robot** — that is what p_g controls.



So "the **stored** action" is *usually* the student's and *occasionally* the teacher's. That small teacher fraction is the only reason Q_g ever gets trained on actions the teacher **really took** — everything else it is asked about is imagined.

How the critic is trained

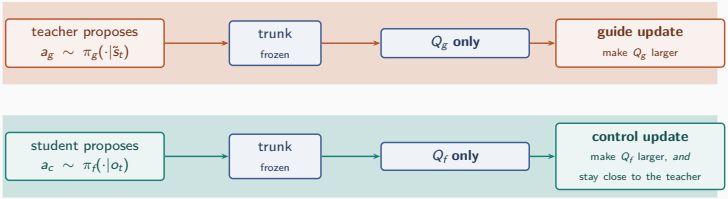
Both head-pairs see the **same** input: the state, and the action that was really executed there. So what makes them different? Only the **bootstrap** — whose policy is assumed to act next.



How does the teacher influence the critic at all? Not through the top row — that is whatever was executed. Through the **bottom** row: Q_g's target is built by asking "what if the *teacher* plays from here?", so the teacher's policy shapes what Q_g is trained *toward*. Q_f asks the same question about the student. Same input, two different targets, two different value functions.

How each actor uses the critic

Now the critic is **frozen** and simply answers questions. Each actor asks *its own* head-pair how good its proposed action is, and moves to make that answer larger.



Frozen = only the *actor's* weights change here.

Two critics, or one? *Two value functions, one network.* Q_f and Q_g are two different *questions* — $\approx$96% of the critic's parameters (the body) are shared and trained by both; only the small output heads are private. The labels say *which head is read*, not which network. Independently of that, the body is duplicated $\times 2$ for pessimism, so 2 questions $\times$ 2 copies = 4 numbers.

Why the teacher needs its own scorecard

A value function is a **scorecard**: it rates a move by assuming somebody keeps playing afterwards. *Who* that somebody is changes the rating.

If the teacher is rated on the student's scorecard Q_f
every move is scored as "*this move, and then the **student** plays on*". The best the teacher can possibly score is *one good move followed by student play* — its advantage is capped at a single step, however much better it really is.

With its own scorecard Q_g
moves are scored as "*this move, and then the **teacher** plays on*". Now the teacher's improvements build on each other, and the gap between teacher and student can actually grow — which is the entire point of having a teacher.

The student keeps using Q_f throughout: it must be judged by what the **blind, deployed** policy can really achieve, not by what the teacher could do.

Objectives

The two value functions, side by side

	Q_f **(student head)**	Q_g **(guide head)**
trained on	executed action a_t	executed action a_t
"and then?"	the student π_f continues	the teacher π_g continues
temperature used	α_f	α_g
so it estimates	$\approx Q^{\pi_f}$	$\approx Q^{\pi_g}$

Each head uses the ordinary SAC target — *the reward that really followed, plus the discounted value of the next state* — and only the policy used to value that next state differs. Distributional critic, n-step returns and the EMA target network are inherited unchanged.

Actors: one new term in the whole method

The guide is **plain SAC**. The student is **plain SAC plus one pull toward the guide** — that single term is the entire method.

Guide — ordinary SAC on privileged input, against its own heads:

$$\mathcal{L}_{\pi_g} \;=\; \mathbb{E}\Big[\, \alpha_g \log \pi_g(a_g|\tilde{s}_t) \;-\; \min_i Q_{g,i}(\tilde{s}_t, a_g) \,\Big]$$

Control — the same objective on proprioception, plus guidance:

$$\mathcal{L}_{\pi_f} \;=\; \mathbb{E}\Big[\, \alpha_f \log \pi_f(a_c|o_t) \;-\; \min_i Q_{f,i}(\tilde{s}_t, a_c) \;+\; \underbrace{\lambda(t)\,\bar{q}\; D_{\mathrm{KL}}\big(\pi_f(\cdot|o_t) \,\big\|\, \mathrm{sg}[\pi_g(\cdot|\tilde{s}_t)]\big)}_{\text{the only term Guided SAC adds}} \,\Big]$$

- **KL, not L_2.** Both policies are tanh-Gaussian, so the KL is closed-form and matches the teacher's *variance*, not just its mean — the student may stay uncertain where the teacher is. The teacher is stop-gradded.
- $\bar{q} = \mathbb{E}|\min_i Q_{f,i}|$ keeps λ dimensionless as values grow; α_f, α_g are learned **separately**.

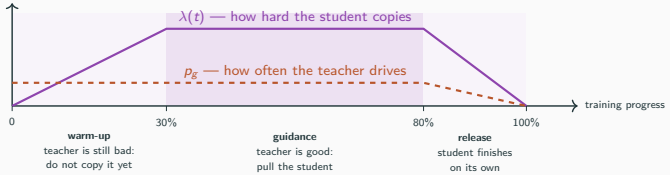
Schedules

Two knobs, doing two different things

They are **independent mechanisms**: one changes a *loss*, the other changes the *data*. Both are **open-loop** — pre-programmed functions of the training step counter. Neither reacts to how training is going, and neither reads the other.

	$\lambda(t)$	p_g
changes	the **student's loss**	**what enters the buffer**
does	pulls the student toward the teacher	lets the teacher *drive* a few % of steps
exists to	transfer the teacher's skill	make Q_g answer about *real* teacher actions
if $= 0$	student ignores the teacher	teacher never acts; Q_g only sees imagined actions
how it changes	step counter: $0 \to \max \to 0$	step counter: constant, then $\to 0$

Both are scheduled, and both stop before the end



Both curves are **read off the step counter** — no feedback, no adaptation. λ starts at **zero** because an unconverged teacher emits noise; both end at zero so the student finishes under its own constraints.

The imitation gap — the method's real weakness

π_g solves the *fully observed* MDP. π_f must solve the *POMDP*. These are different problems, and projecting one onto the other is not optimal in general.

Concretely, for blind locomotion

A teacher that sees the height scan never learns to probe the ground with a foot, never steps cautiously into the unknown, never moves to enrich its own IMU signal — it has no need to. Imitating it can therefore *suppress exactly the behaviours that keep a blind robot alive*.

Mitigations built in, weakest to strongest:

1. **KL rather than L_2** — the student may stay uncertain where the teacher is.
2. $\lambda \to 0$ — guidance is removed before training ends.
3. **Latent supervision** (planned): supervise a learned *representation* $\hat{z} = E(o_t)$ against privileged z and let the policy decide how to use it — closes the gap by construction.

Deployment

Deployment — what survives

discarded: guide actor π_g • critic Q_f, Q_g • target critic • temperatures α_f, α_g • replay buffer • λ, p_g

- The deployed artifact is **architecturally identical** to a vanilla SAC actor. Guided training costs *nothing* at inference — the strongest practical argument for asymmetric methods.
- Nothing privileged can leak: the actor consumes a fixed 285-dim slice and the export path never reads the guide's files.

Implementation

Code map

Built as an **extension** of the FlashSAC backbone, never a fork: the vendored learning core is byte-identical, and without the `--guided` flag the pipeline is the unmodified baseline.

`gsac/config.py`	config dataclass; $\lambda(t)$, $p_g(t)$; fail-fast validation
`gsac/network.py`	`GuidedDoubleCritic`: shared trunks + second head-pair
`gsac/update.py`	the three losses above, each traced to its source equation
`gsac/agent.py`	`GuidedFlashSACAgent`: networks, rollout mixing, updates
`train.py --guided`	launcher flag; everything guided is behind it

The subclass overrides `sample_actions` (mixing), `update` (the three losses) and `save`/`load` (guide checkpoints); everything else is inherited. The teacher's action a_g is never stored — it is recomputed from the sampled batch at update time, so teacher/state alignment holds by construction.

One codebase, four experiments

Each rung switches on *one* mechanism, so any difference in outcome can be attributed to it:

	critic sees	**guide trained?**	**guide acts?**	**imitation**	
A1	o_t (blind)	no	no	no	plain SAC
A2	$\tilde{s}_t$ (privileged)	no	no	no	privileged critic (**M1**)
A4	$\tilde{s}_t$	yes	$p_g > 0$	no ($\lambda = 0$)	… and the teacher drives (**M2**)
A5	$\tilde{s}_t$	yes	$p_g > 0$	yes ($\lambda > 0$)	… and it advises (**M3**) = full

A1 $\rightarrow$ **A2** is not "no teacher vs teacher" — neither rung has a second network. It changes only what the *critic* may see.

A4, "the teacher acts but never advises": it drives a few % of steps so Q_g is trained on real teacher actions, but $\lambda = 0$ leaves the student unpulled — separating the value of the teacher's *data* from that of its *advice*. (A3, latent supervision, is a separate future branch, not a step on this line.)

Summary

- Blind locomotion is a genuine POMDP; the simulator knows far more than the robot ever will.
- Guided SAC bundles three separable mechanisms. We keep the free one (asymmetric critic), soften the risky one (imitation via scheduled KL), and use the expensive one sparingly and for a different reason than the original (value grounding).
- The single-agent reduction is not a copy: the teacher needs **its own value head**, otherwise its advantage is capped by construction.
- The imitation gap is the method's real theoretical weakness; latent supervision is the principled successor if action-space imitation does not pay.
- **Deployment cost is exactly zero** — the shipped network is a plain SAC actor on proprioception.